Activity A.2. Basic Shapes in OpenSCAD

Activity A.2. Basic Shapes in OpenSCAD

Download this activity

Now that we have tried out the workflow for creating a square, we can try out other built-in 2D shapes as well as adding, subtracting or intersecting two or more shapes. We used a square the most straightforward way, by just specifying a side. OpenSCAD also lets you create "squares" (rectangles, really) with sides that are different lengths. You can also create circles, and general polygons.

Steps:

All the activities here assume that OpenSCAD is open and working, and that you have launched it with the usual graphical user interface.

- Go to the File menu, and select New File.
- Navigate to the Editor window.
- In the Editor window type the code to create an object (code given in examples that follow).
- In the main Menu bar along the top, select Save and give the model an appropriate name ending in .scad, like rectangle.scad.

Rendering and Exporting an SVG

The file ending in .scad can be edited in OpenSCAD going forward. (SVG files can be opened in OpenSCAD, but not edited.) Now that we have saved the file we can try rendering, exporting and printing it.

- In the Design menu, click Render. This creates an exportable version of the file.
- In the File menu, select Export and then Export as SVG.
- The file should now be exported to a location you specify and ready for tactile printing. It should produce the shapes as described in what follows.

Code to Create General 2D Shapes

The basic 2D shapes in OpenSCAD are squares, circles and polygons. With various tricks we can generate many types of shapes from these. Let's look at

generating a few common shapes from these options. We already saw how to draw a square in the previous activity. Suppose we want a rectangle instead? We would define a "square" with different length and width by typing something like this in the Editor window:

```
square([25, 100]);
```

creates a rectangle 25 mm long and 100 mm high.

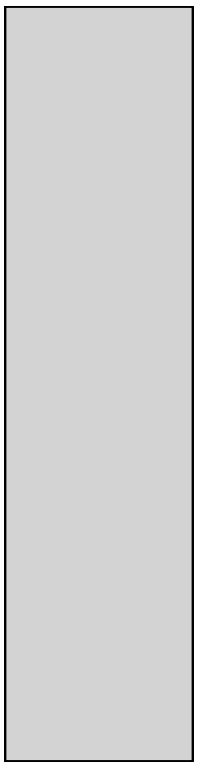

Figure 1: A rectangle 25 by 100, file square25,100.svg

A rectangle 25 by 100, file square25,100.svg

A circle is defined several different ways. One is to define the radius, like this:

```
circle(r = 25);
```

which would give us a circle of radius 25 mm. In fact we can just say:

```
circle(25);
```

which accomplishes the same thing. (There are other ways to specify a circle, but we will leave you with just these two.)

A circle of 25 mm radius, file circle25.svg

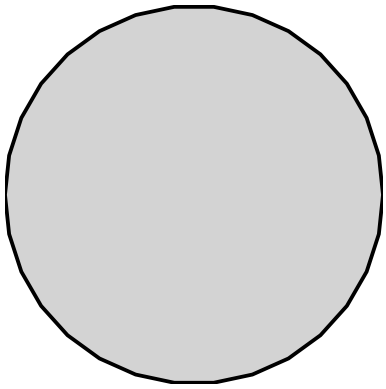

Figure 2: A circle of 25 mm radius, file circle25.svg

Suppose instead we wanted a regular polygon - that is, a shape with N sides of equal length. A "circle" in OpenSCAD is really a lot of tiny straight lines, called facets. With enough facets, every line is short enough that they seem to be making a continuous curve. But if instead we said, "draw a circle but with only three facets" we would get a triangle, like this:

```
circle(r = 25, $fn = 3);
```

where $fn = 3 tells us that the number of facets is 3, and the radius is the distance from the center of the shape to a vertex. (This is called the "inscribed polygon" of the circle of the same radius; it is the biggest polygon that can be drawn that will fit entirely inside the circle.) If $fn = 5, we would get a pentagon, and so on.

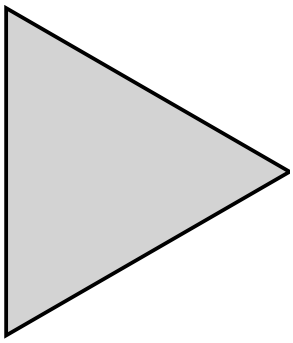

Figure 3: A "circle" turned into a triangle of radius 25, file radius25.svg

A "circle" turned into a triangle of radius 25, file radius25.svg

Teaching Tip: Experiment

Encourage students to experiment with slight changes in numbers or parameters (try a different number for the radius or a different number of facets). Use these small adjustments to build confidence and explore cause/effect relationships in the code.

Other, more general polygons can be drawn by creating a list of points that define its vertices. For example, to draw a triangle whose vertices are the points (0, 0), (100, 0), and (50, 100) we would write:

```
polygon(points = [ [0, 0], [100, 0], [50, 100] ]);
```

Note that each (x, y) coordinate pair is enclosed in a pair of square brackets, and the whole set is inside another set of square brackets and a pair of parentheses. Be careful not to mix those up since they mean different things in OpenSCAD.

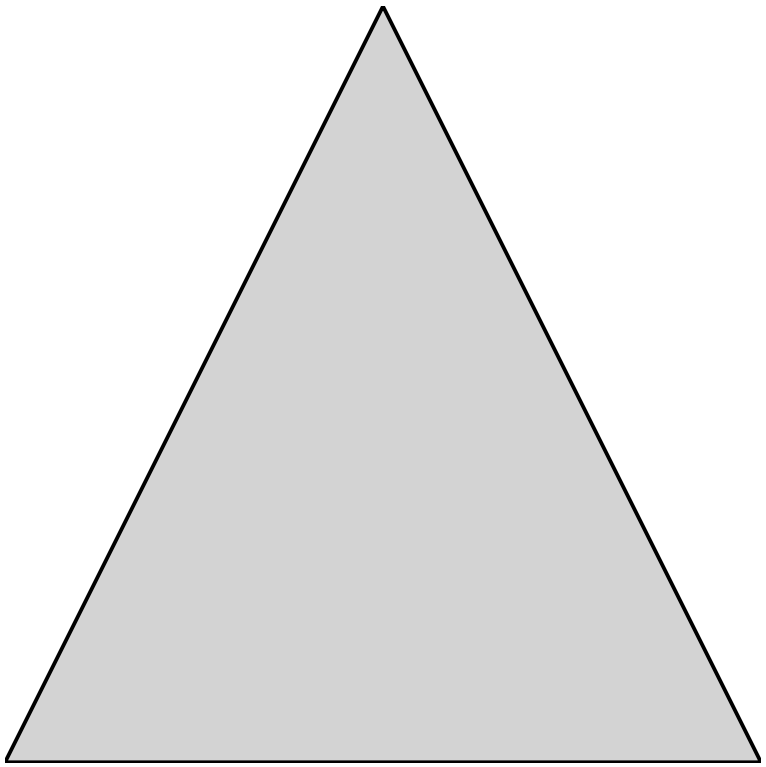

Figure 4: A triangle created with the polygon function, file polygon.svg

A triangle created with the polygon function, file polygon.svg

This polygon can be as complex as we want. By default the first and last points in the list will be connected. There are many more options and tricks in the OpenSCAD manual "2D Subsystem" page.

Note: the images above are embedded from the original site with empty alt text — worth writing real descriptive alt text when you migrate the actual image files over.

Previous: Activity A.1: Create a Square · **Next:** Activity A.3: Rotating, Translating, Scaling · **Back to:** Adventure Map · Download all TADA PDFs